```
[root@legacy ~]# strace -c ls
anaconda-ks.cfg       cachegrind.out.6941          install.log.syslog
bonnie++-1.03e.tar    Desktop                      iptables-squid-email
cache1.c              Documents                    Music
cache2.c              Download                     myrules
cachegrind.out.6723   fping-2.4b2-9.fc11.i586.rpm  nagios-3.0.6
cachegrind.out.6831   install.log                  nagios-3.0.6.tar
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 89.37    0.000280         280         1           execve
 10.83    0.000034           1        26           mmap2
  0.00    0.000000           0        12           read
  0.00    0.000000           0         6           write
  0.00    0.000000           0        13           open
  0.00    0.000000           0        15           close
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         2           ioctl
  0.00    0.000000           0         4           munmap
  0.00    0.000000           0         1           uname
  0.00    0.000000           0         7           mprotect
  0.00    0.000000           0         2           rt_sigaction
  0.00    0.000000           0         1           rt_sigprocmask
  0.00    0.000000           0         1           getrlimit
  0.00    0.000000           0        13           fstat64
  0.00    0.000000           0         2           getdents64
  0.00    0.000000           0         1           fcntl64
  0.00    0.000000           0         2         1 futex
  0.00    0.000000           0         1           set_thread_area
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         1           statfs64
  0.00    0.000000           0         1           set_robust_list
------ ----------- ----------- --------- --------- ----------------
100.00    0.000314                   117         2 total
```

Figure 2: Tracing ls with strace

```
[root@legacy ~]# strace -c -o lsoutput.txt ls
anaconda-ks.cfg       cachegrind.out.6941          install.log.syslog
bonnie++-1.03e.tar    Desktop                      iptables-squid-email
cache1.c              Documents                    lsoutput.txt
cache2.c              Download                     Music
cachegrind.out.6723   fping-2.4b2-9.fc11.i586.rpm  myrules
cachegrind.out.6831   install.log                  nagios-3.0.6
[root@legacy ~]# cat lsoutput.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 94.26    0.000476          79         6           write
  5.74    0.000029           2        13           fstat64
  0.00    0.000000           0        12           read
  0.00    0.000000           0        13           open
  0.00    0.000000           0        15           close
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         2           ioctl
  0.00    0.000000           0         4           munmap
  0.00    0.000000           0         1           uname
  0.00    0.000000           0         7           mprotect
  0.00    0.000000           0         2           rt_sigaction
  0.00    0.000000           0         1           rt_sigprocmask
  0.00    0.000000           0         1           getrlimit
  0.00    0.000000           0        26           mmap2
  0.00    0.000000           0         2           getdents64
  0.00    0.000000           0         1           fcntl64
  0.00    0.000000           0         2         1 futex
  0.00    0.000000           0         1           set_thread_area
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         1           statfs64
  0.00    0.000000           0         1           set_robust_list
------ ----------- ----------- --------- --------- ----------------
100.00    0.000505                   117         2 total
```

Figure 3: Redirecting the output of strace to a text file

Step 3 Configure hugetlbfs if needed by an application. Mount it on any location and make it permanent. Append the following line to your /etc/fstab file:

```
none    /hugepage       hugetlbfs   defaults 0 0
```

Run *mount -a* to activate the changes.

Now, if we cat the /proc/meminfo file, we can see the hugepages are activated in the system and can be used by any application that needs them:

```
[root@legacy ~]# cat /proc/meminfo | grep Huge
HugePages_Total:    10
HugePages_Free:     10
HugePages_Rsvd:      0
Hugepagesize:     4096 kB
```

Configuring the TLB increases the chances of the TLB cache being hit more frequently, and significantly reduces page table entry (PTE) visit count.

Viewing system calls

Many a time, after devoting a good amount of time downloading and compiling a program, it simply refuses to run. We are left with the last resort of tracing the output of the program that's misbehaving. Tracing the output of a program throws up a lot of data that is not usually available when the program is run normally.

strace comes in handy for tracing the system calls of a program. When it is run in conjunction with a program, it outputs all the calls that the program makes to the kernel. A program might often fail to initiate because of insufficient memory. Tracing the output of the program will let us pinpoint the exact issue.

To understand this better, take a look at Figure 2, which shows the output thrown by *strace* when used along with the ls command. As you can see, it displays all the system calls made by ls, along with the number of times each one was made. You can also redirect the output of *strace* by using the *-o* option as shown in Figure 3.

By now you must have realised how *strace* can be very handy in investigating locks, identifying problems caused by improper permissions and also identifying IO problems.

Tuning memory caches

In addition to the hardware cache, there are other types of caches in the Linux kernel—implemented in the main memory. The idea behind it is to prevent the kernel from having to read the information on the hard disk as much as possible, thereby reducing latency.

The most important of them all is page cache. It plays a very pivotal role in disk IO.

Demand paging

The kernel delays the memory allocation until a process asks for it for performance reasons. Due to the locality of reference, the processes do not use the whole memory they had asked for at one time. Just think about it -- while executing a large program, it is more likely that only a small part of the executable image is run at a given time. So it will be a total waste of memory to allocate space for the full executable image.

Linux delays the allocation of memory until it is asked for by the process. This process of delaying the memory allocation for a process until the last minute is known as demand paging.